Desarrollo propio con KMNIST

CNN en PyTorch teniendo en cuenta la revisión del script del profesor

Taller 3 – Punto 4 – Informe de ejecución

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiantes: Juan Felipe Rodríguez Galindo – 20261595004
Programa: Maestría en Ciencias de la Información y las Comunicaciones
Fecha: 10 de octubre de 2026

Universidad Distrital Francisco José de Caldas

Facultad de Ingeniería – Bogotá D.C., Colombia

1. Qué se hizo y cómo se relaciona con el punto 3

Se repitió el flujo del cuaderno del profesor con **KMNIST** (70 000 imágenes de 28×28 de hiragana cursivo, 10 clases), se completaron los cinco bloques `TODO` del cuaderno propuesto y se respondieron sus tres preguntas. La revisión del punto 3 se usó como guía: la tabla 1 indica qué se conservó y qué se cambió o agregó.

Tabla 1: Qué se conservó del script y qué se cambió o agregó.

Decisión	Según la revisión del script (punto 3)	Qué se hizo en el desarrollo propio
Conservado	Partición 55 000 / 5 000 / 10 000, normalización solo con estadísticos del entrenamiento, prueba usada una vez	Igual; la prueba solo informa el resultado final
Conservado	Puntos de control `verificar()`, Adam con $\eta = 10^{-3}$, lotes de 64	Igual: 11 verificaciones del taller y 9 propias
Cambiado	CNN sin relleno ni *BatchNorm* (12 810 parámetros)	`CNNPropia`: relleno 1, *BatchNorm* y *dropout* 0,3; **20 586** parámetros
Cambiado	Comparaciones no pareadas, una sola corrida	Cargador de datos por corrida y **5 semillas** (aumento) o 3 (ablación y bases)
Cambiado	Paso 5 con 3 tasas, 2 épocas y sin control	5 tasas de 10^{-1} a 10^{-5} y un control sin *BatchNorm*
Cambiado	MLP no evaluado en prueba	MLP y CNN del docente evaluados en prueba (88,44 % y 91,20 %)
Cambiado	Solo MNIST	KMNIST como conjunto principal; FashionMNIST y MNIST con la misma red

2. Ejecución

El cuaderno `Taller_3_CNN_KMNIST.ipynb` se ejecutó **de principio a fin, sin errores**, con las salidas visibles, en CPU (Apple M1, PyTorch 2.14.1). Cada bloque del taller termina en un punto de control automático; los 11 del taller y 9 propios pasaron (`[OK]`).

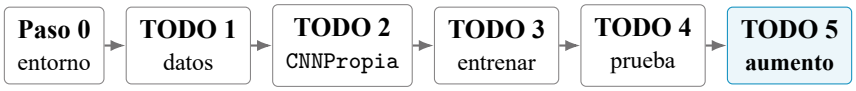


Figura 1: Bloques ejecutados; todos pasaron su punto de control.

Tabla 2: Puntos de control del cuaderno. Los 11 primeros son los del taller; además hay 9 verificaciones propias.

Bloque	Verificación	Valor observado	Estado
TODO 1	El conteo por clase suma las imágenes de entrenamiento	55 000 (razón máx/mín 1,015)	OK
TODO 2	Salida de forma $(4, 10)$	$(4, 10)$	OK
	Menos de 200 000 parámetros	20 586	OK
	Usa `BatchNorm2d` y `Dropout`	ambos presentes	OK
	Al menos 2 capas convolucionales	2	OK
TODO 3	La pérdida de entrenamiento baja entre la primera y la última época	0,374 → 0,117	OK
	Exactitud de validación > 80 %	97,40 %	OK
TODO 4	El par con más errores es el máximo fuera de la diagonal	su→tsu, 60 veces	OK
	La suma de la matriz de confusión es el tamaño de prueba	10 000	OK
TODO 5	El aumento conserva la forma del lote	$(8, 1, 28, 28)$	OK
	El aumento modifica el lote en algunas llamadas	sí (30 pruebas)	OK
Extras	Parámetros = tabla, validación > 90 %, aumento por muestra, redes del docente (12 810 y 101 770 parámetros), modelo guardado y recargado	9 de 9	OK

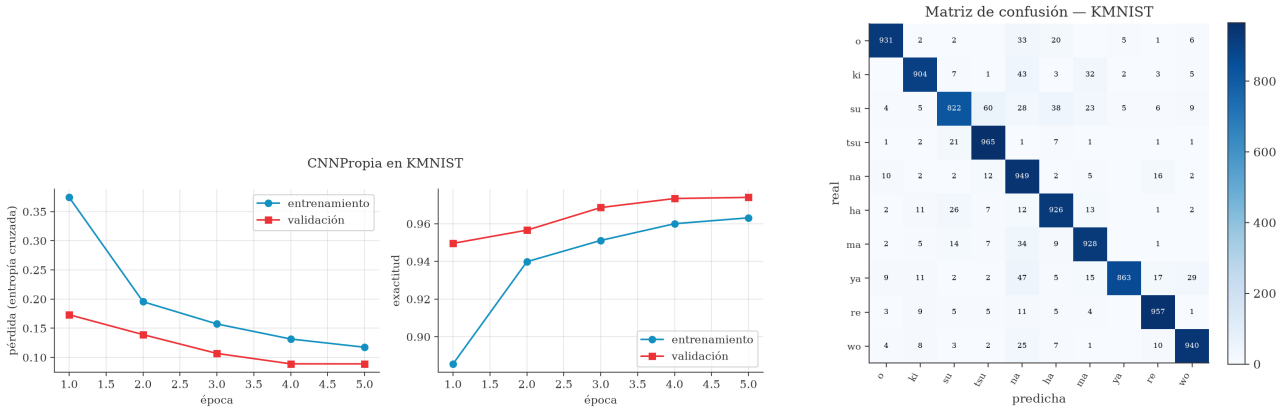


Figura 2: Salidas del cuaderno: pérdida y exactitud por época (5 épocas, 49 s) y matriz de confusión en prueba (815 errores de 10 000).

Resultado. `CNNPropia` **(20 586 parámetros) alcanza 97,40 % de exactitud en validación y 91,85 % en prueba. La prueba de KMNIST es más difícil que la validación (5,5 pts), diferencia que no aparece en MNIST ni en FashionMNIST.**

3. Las tres preguntas del taller

3.1. ¿Por qué cada componente y qué se esperaba de él?

Tabla 3: Componentes de la red: qué se esperaba y qué se midió.

Componente	Se esperaba	Qué se midió	Resultado
Convolución	Rasgos locales con pocos parámetros	La CNN del docente (12 810 parámetros) supera al MLP (101 770): 91,20 % frente a 88,44 % en prueba	Confirmado
BatchNorm (estabilidad)	Entrenar de forma más estable	Con $\eta = 0{,}1$ la misma red llega a 82,46 % con *BatchNorm* y a 9,76 % sin ella	Confirmado
BatchNorm (exactitud)	Mejor exactitud final	Sin ella, la prueba cambia 0,54 pts (dentro del ruido)	No confirmado
Dropout	Regularizar la capa final	Sin él: $-0{,}52$ pts en prueba y brecha de 1,45 frente a 0,90 pts	Confirmado
ReLU y *max-pooling*	No linealidad y tolerar desplazamientos	No se ablacionaron por separado	No evaluado

3.2. ¿Por qué se confunde el par de clases con más errores?

El par más frecuente es **su** → **tsu** (60 veces; en sentido contrario, 21). *su* es la clase más difícil (82,2 %). Al mirar imágenes concretas (figura 3), los *su* mal clasificados son casi siempre una **variante cursiva** distinta de la dominante en la clase y se parecen a grafías de *tsu* bien clasificadas: hay variación **dentro** de la clase, no desbalance (las clases tienen entre 5 450 y 5 533 imágenes). La confusión es asimétrica, y *na* actúa como "atractor" de *ya*, *ki* y *ma*. Es una lectura visual de las imágenes, no una medición.

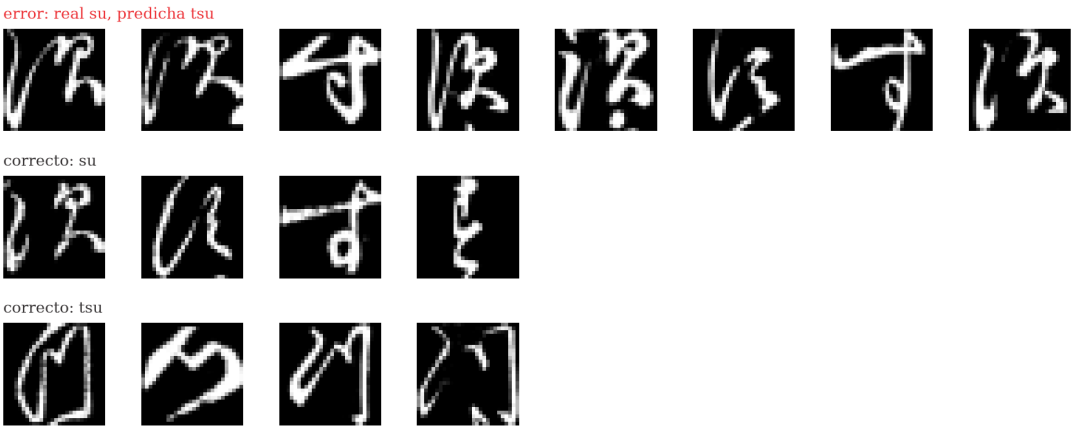


Figura 3: Arriba, errores *su* leído como *tsu*; abajo, aciertos de cada clase para contraste.

3.3. ¿Mejoró el aumento de datos? ¿Cambió la brecha entre entrenamiento y validación?

No mejoró la exactitud de validación: con 5 semillas pareadas, el desplazamiento por lote cambia la validación 0,00 pts y el desplazamiento por muestra 0,12 pts; en prueba fueron −0,33 pts y −0,25 pts. Con 15 épocas el resultado es el mismo (−0,19 pts en prueba). **La brecha sí cambió**: la diferencia entre exactitud de entrenamiento y de validación baja de 0,90 a unos 0,2 puntos, es decir, el aumento regulariza, pero esa menor memorización no se traduce en mejor generalización. Posibles razones, no probadas por separado: a 5 épocas casi no hay sobreajuste que corregir; convolución más *max-pooling* ya tolera desplazamientos pequeños; `torch.roll` es circular; y los errores de prueba parecen venir de variantes de escritura y no de traslaciones.

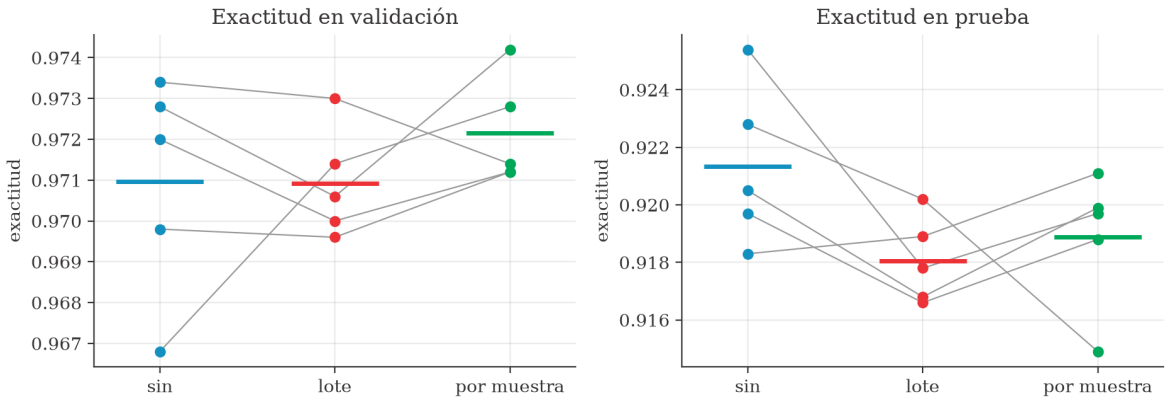


Figura 4: Exactitud por semilla (cada punto es una semilla; las líneas grises unen la misma semilla) sin aumento, por lote y por muestra.

4. Lectura honesta y reproducibilidad

- **Validación y prueba no son intercambiables en KMNIST** (97,40 % frente a 91,85 %). No es un error; la prueba es más difícil y la causa exacta no se midió. Las conclusiones se apoyan en la prueba.
- **Resultado negativo:** el aumento por desplazamiento no mejoró la exactitud; se reporta tal cual. **Mejora clara:** un tercer bloque convolucional (2,11 pts en prueba con 8 704 parámetros más).
- **Limitaciones:** 3 a 5 semillas, 5 épocas, una arquitectura base y una partición de validación fija; las diferencias de décimas de punto no son concluyentes.
- **Reproducir:** `make notebook` (aprox. 1 h en CPU; reutiliza los resultados guardados) o `make rapido` (solo los 5 `TODO`, unos 3 min). Partición con semilla 42; comparaciones con semillas 0 a 4 (aumento) y 0 a 2 (ablación y bases). El análisis completo está en `informe/main.pdf`.